Hart, Noah - u1537288 - Actively Contributing

Level Design

This sprint, I focused on refining the overall level design and communicating the basic layout to our environment artist so they could begin assembling assets in a white-box stage. Our goal is to create reusable templates for the train cars that minimize the workload for the environment team while still supporting procedural asset placement to keep each playthrough varied. Over Fall Break, I plan to fully document the level design process and the general gameplay loop of train cars and train stations so that all of this information is centralized and easy to reference. I also want to revisit and streamline the technical design document while exploring websites used by other teams for clearer organization.

Engineering

On the engineering side, I implemented an **Encounter Manager** that randomizes encounters from one train car to the next. Currently, we've defined three encounter types:

- **Elimination**: Defeat all enemies.
- **Survival**: Stay alive for a set amount of time.
- **Capture**: Hold a zone while fending off enemies.

The manager is currently integrated with the level and level blueprint, opening the train door once conditions are met. Beyond encounter tracking, it also selects a random **modifier** and/or **challenge** for each room. A weighting system ensures the possibility of having a modifier, a challenge, both, or neither. Each modifier, challenge, and encounter is structured in its own script, making the system scalable and easy to expand in future iterations, just by adding new scripts for the respective type.

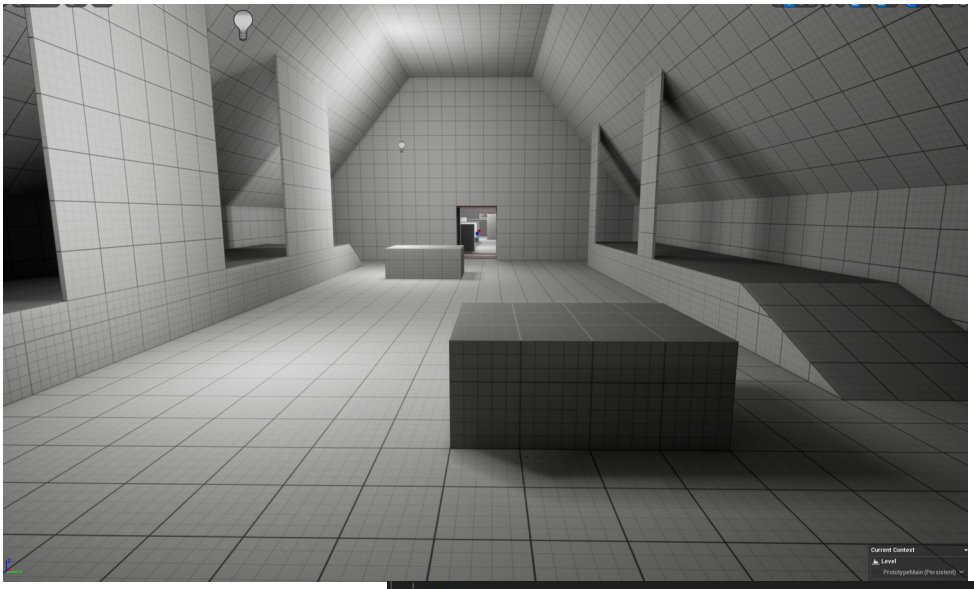
Looking Forward

The next steps are to:

1. **Randomize the train sequence** so each run feels fresh.
2. **Set up triggers** (either via box volumes or level loads) to initiate encounters and apply their respective modifiers and challenges.

3. **Outline train station layouts**, being able to use the same level template while achieving variety through asset placement, and environmental storytelling.

These priorities will establish the foundation for procedural-level flow and ensure that encounters feel dynamic and replayable while keeping asset creation efficient.



EncounterInterface.cpp
EncounterManager.cpp
KillEnemiesEncounter.cpp

```
public:
    AEncounterManager();

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Encounters")
    Changed in 0 Blueprints
    TArray<TSubclassOf<AActor>> EncounterPool;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Modifiers")
    Changed in 0 Blueprints
    TArray<FWeightedOption> ModifierPool;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Challenges")
    Changed in 0 Blueprints
    TArray<FWeightedOption> ChallengePool;

    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Train")
    Changed in 0 Blueprints
    int32 MinEncountersPerTrain = 3;

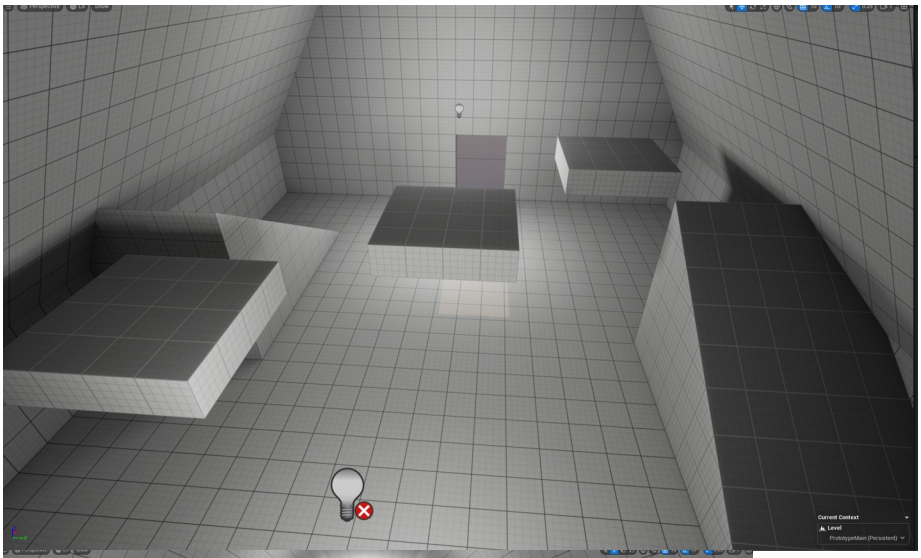
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Train")
    Changed in 1 Blueprint
    int32 MaxEncountersPerTrain = 4;

    UPROPERTY(BlueprintAssignable, Category = "Events")
    Changed in 0 Blueprints
    FOnEncounterComplete OnEncounterComplete;

    UFUNCTION(BlueprintCallable, Category = "Train")
    0 Blueprint references
    void StartTrainPhase();

    UFUNCTION(BlueprintCallable, Category = "Train")
    0 Blueprint references
    void StartNextEncounter();

protected:
    virtual void BeginPlay() override;
    virtual void Tick(float DeltaTime) override;
```


Current Context
Level
PrototypeMain (Persistent)


Current Context
Level
PrototypeMain (Persistent)